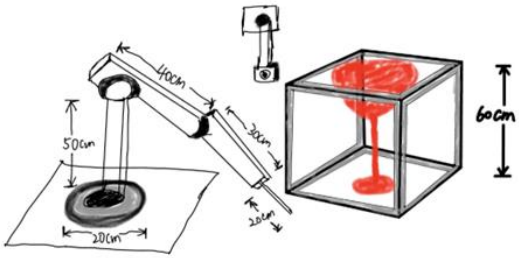
ROBOT MANIPULATOR

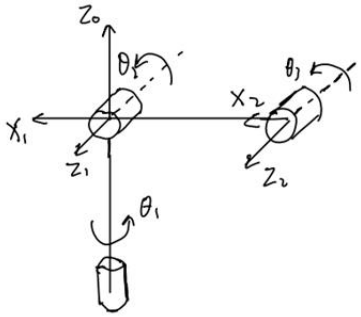
STEP #2

Zejun Yin
251388005

Forward Kinematics

To generate the terminal position, a. DH table should be applied. From the SCARA manipulator in Step 1, as shown in the following. It can be regarded as the combination of three rotational joints. The frame of each joint is established in the figure. From this frame we can easily get the DH table, which is demonstrated below. We can get three homogeneous matrices and the forward kinematic is then calculated. Then we can define a matlab function, where there are three inputs since the d1, $\alpha 2$, $\alpha 3$ are fixed and only $\theta 1$,$\theta 2$ and $\theta 3$ will change the final position. The matlab code is attached in the appendix. To verify the function, three different initial inputs of θ vectors are tested, and the outputs are in the following figures.






```
theta1: 30.00, theta2: 45.00, theta3: 60.00
T1
    0.6330   -0.7544    0.1736   56.0073
    0.1116   -0.1330   -0.9848    9.8756
    0.7660    0.6428         0   86.6621
         0         0         0    1.0000

theta1: 10.00, theta2: 20.00, theta3: 30.00
T2
    0.6330   -0.7544    0.1736   56.0073
    0.1116   -0.1330   -0.9848    9.8756
    0.7660    0.6428         0   86.6621
         0         0         0    1.0000

theta1: 45.00, theta2: 45.00, theta3: 45.00
T3
         0   -0.7071    0.7071   20.0000
         0   -0.7071   -0.7071   20.0000
    1.0000         0         0  108.2843
         0         0         0    1.0000
```

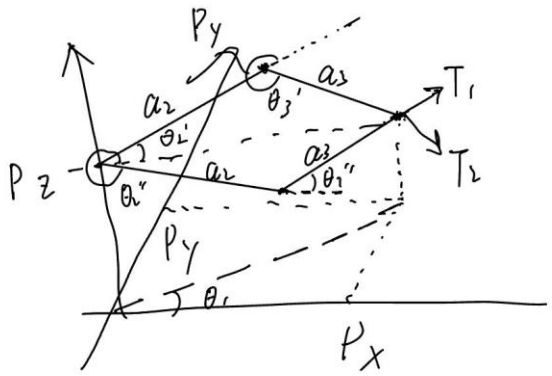
Inverse Kinematics

As for the IK, the situation is more complicated. There is the final position T as the input, and the corresponding angles of θs are to be determined. After analyzing the manipulator model, It is straightforward to conclude that θ1 is solely depended on the X and Y ccordinate, and θ1= Atan2 (xc, yc) unless xc = yc = 0. Next, for θ2 and θ3, in figure we can see that the θ2 and θ3 satisfy

r = a2*cos (θ2) + a3 *cos (θ2 + θ3)

z = d1 + a2 *sin (θ2) + a*3sin (θ2 + θ3)

where r^2=px^2+py^2



Also, from figure we can see that usually there are two possible positions, T1 and T2. One is elbow up and the other is elbow down. The specific situation depends on the actual final state. Since there is no orientation at the end tool, which means one location will only have up to two situations, the elbow up or elbow down is easy to determine: Maybe both are OK or maybe only one of them is available because there are obstacles around the manipulator. The input of matlab function should be a terminal matrix T, and the outputs will contain three angles θ 1, θ2, θ3. To verify the validity of the function, I used the three outputs matrices of forward kinematics, where the initial θs have known from the beginning. And the test results are as follow. It turns out that the inverse kinematics can get the correct θs successfully.

```
T3
         0   -0.7071    0.7071   20.0000
         0   -0.7071   -0.7071   20.0000
    1.0000         0         0  108.2843
         0         0         0    1.0000

theta1: 45.00, theta2: 45.00, theta3: 45.00
```

```
T2
    0.6330   -0.7544    0.1736   56.0073
    0.1116   -0.1330   -0.9848    9.8756
    0.7660    0.6428         0   86.6621
         0         0         0    1.0000

theta1: 10.00, theta2: 20.00, theta3: 30.00
```

```
T1
   -0.2241   -0.8365    0.5000   17.7706
   -0.1294   -0.4830   -0.8660   10.2598
    0.9659   -0.2588         0  107.2620
         0         0         0    1.0000

theta1: 30.00, theta2: 45.00, theta3: 60.00
```

Differential kinematics

The Jacobian matrix is essential in describing the relationship between joint velocities and the Cartesian velocities of the end-effector. In this project, the SCARA robot with joint angles θ 1, θ2 and θ3, the forward kinematics provides the position (x, y, z) of the end-effector as:

x = (a2 *cos (θ2) + a3*cos (θ2 + θ3)) *cos (θ1)

y = (a2 *cos (θ2) + a3*cos (θ2 + θ3)) *sin (θ1)

z = d1 + a2 *sin (θ2) + a3 *sin (θ2 + θ3)

To derive the Jacobian matrix, we calculate the partial derivatives of (x, y, z) with respect to each joint angle, from lecture slice:

Solution: Manipulator's Jacobian

$$J = \begin{bmatrix} -a_2 s_{\theta_1} c_{\theta_2} - a_3 s_{\theta_1} c_{(\theta_2+\theta_3)} & -a_2 s_{\theta_2} c_{\theta_1} - a_3 c_{\theta_1} s_{(\theta_2+\theta_3)} & -a_3 c_{\theta_1} s_{(\theta_2+\theta_3)} \\ a_2 c_{\theta_1} c_{\theta_2} + a_3 c_{\theta_1} c_{(\theta_2+\theta_3)} & -a_2 s_{\theta_2} s_{\theta_1} - a_3 s_{\theta_1} s_{(\theta_2+\theta_3)} & -a_3 s_{\theta_1} s_{(\theta_2+\theta_3)} \\ 0 & a_2 c_{\theta_2} + a_3 c_{(\theta_2+\theta_3)} & a_3 c_{(\theta_2+\theta_3)} \end{bmatrix}$$

To verify the correctness of this Jacobian matrix, we implemented a MATLAB function that calculates J based on the given joint angles and link lengths. Testing was conducted using different joint angle configurations to ensure the matrix accurately describes the relationship between joint velocities and end-effector velocities. And the results are:

```
Joint velocities:
    0.1000
    0.2000
    0.3000

End-effector velocity vector:
  -14.9983
    3.1303
   17.1594
```

```
Joint velocities:
    0.1000
    0.3000
    0.9000

End-effector velocity vector:
  -32.1881
    0.0992
   34.4167
```

```
Joint velocities:
    0.3000
    0.3000
    0.3000

End-effector velocity vector:
  -20.5839
   13.6951
   22.8465
```

Appendix

FK:

theta1 = 30;

theta2 = 45;

theta3 = 60;

```
T3 = forwardkin(theta1, theta2, theta3);
disp(sprintf('theta1: %.2f, theta2: %.2f, theta3: %.2f', theta1, theta2, theta3));
disp('T');
disp(T3);

function T = forwardkin(theta1, theta2, theta3)
  % Define the twist angle α in D-H parameters, using degrees directly
  alpha1 = 90;
  alpha2 = 0;
  alpha3 = 0;

  % Define the D-H parameter transformation matrices, using sind and cosd to handle angles
  A1 = [cosd(theta1), -sind(theta1) * cosd(alpha1), sind(theta1) * sind(alpha1), 0;
      sind(theta1), cosd(theta1) * cosd(alpha1), -cosd(theta1) * sind(alpha1), 0;
      0, sind(alpha1), cosd(alpha1), 50;
      0, 0, 0, 1];

  A2 = [cosd(theta2), -sind(theta2) * cosd(alpha2), sind(theta2) * sind(alpha2), 40 * cosd(theta2);
      sind(theta2), cosd(theta2) * cosd(alpha2), -cosd(theta2) * sind(alpha2), 40 * sind(theta2);
      0, sind(alpha2), cosd(alpha2), 0;
      0, 0, 0, 1];

  A3 = [cosd(theta3), -sind(theta3) * cosd(alpha3), sind(theta3) * sind(alpha3), 30 * cosd(theta3);
```

```
        sind(theta3), cosd(theta3) * cosd(alpha3), -cosd(theta3) * sind(alpha3), 30 *
sind(theta3);
        0, sind(alpha3), cosd(alpha3), 0;
        0, 0, 0, 1];

    % Calculate the overall transformation matrix
    T = A1 * A2 * A3;
end
```

IK:

```
clear all; close all; clc;

T1=[   -0.2241   -0.8365    0.5000   17.7706;
  -0.1294   -0.4830   -0.8660   10.2598;
   0.9659   -0.2588        0  107.2620;
        0        0        0    1.0000];

[theta1, theta2, theta3] = invkin(T1);
disp('T1');
disp(T1);
disp(sprintf('theta1: %.2f, theta2: %.2f, theta3: %.2f', theta1, theta2, theta3));

function [theta1, theta2, theta3] = invkin(T)
    % Known constants
    a1 = 50; % Link length a1
    a2 = 40; % Link length a2
    a3 = 30; % Link length a3
```

```
    % Extract target position
    px = T(1, 4);
    py = T(2, 4);
    pz = T(3, 4);

    % Calculate theta1
    theta1 = atan2d(py, px);

    % Calculate theta2 and theta3
    r = sqrt(px^2 + py^2); % Projection distance of the end-effector on the x-y plane
    s = pz - a1;          % Displacement of the end-effector along the z direction

    % Use cosine law to calculate theta3
    D = (r^2 + s^2 - a2^2 - a3^2) / (2 * a2 * a3);
    theta3 = atan2d(sqrt(1 - D^2), D); % One of the two solutions, you can choose a different solution if needed

    % Calculate theta2
    theta2 = atan2d(s, r) - atan2d(a3 * sind(theta3), a2 + a3 * cosd(theta3));
end
```

DK:

```
q1 = [10, 20, 30];

J1 = Jacob(q1);
```

```
% Define joint angles and joint velocities      % Current joint angles (can be any value)
qdot = [0.3; 0.3; 0.3];   % Joint velocities

% Calculate end-effector velocity
v = J1 * qdot;

% Output joint velocities
disp('Joint velocities:');
disp(qdot);

% Output end-effector velocity
disp('End-effector velocity vector:');
disp(v);

function J = Jacob(q)
   % Given constants
   a1 = 50; % Link length a1
   a2 = 40; % Link length a2
   a3 = 30; % Link length a3

   % Joint angles
   theta1 = q(1);
   theta2 = q(2);
   theta3 = q(3);

   % Use forward kinematics to calculate the position of each joint
```

```
    % End-effector position
    x = a2 * cosd(theta1) * cosd(theta2) + a3 * cosd(theta1) * cosd(theta2 + theta3);
    y = a2 * sind(theta1) * cosd(theta2) + a3 * sind(theta1) * cosd(theta2 + theta3);
    z = a1 + a2 * sind(theta2) + a3 * sind(theta2 + theta3);

    % Calculate each column of the Jacobian matrix
    % Partial derivative with respect to theta1
    J1 = [-a2 * sind(theta1) * cosd(theta2) - a3 * sind(theta1) * cosd(theta2 + theta3);
          a2 * cosd(theta1) * cosd(theta2) + a3 * cosd(theta1) * cosd(theta2 + theta3);
          0];

    % Partial derivative with respect to theta2
    J2 = [-a2 * cosd(theta1) * sind(theta2) - a3 * cosd(theta1) * sind(theta2 + theta3);
         -a2 * sind(theta1) * sind(theta2) - a3 * sind(theta1) * sind(theta2 + theta3);
          a2 * cosd(theta2) + a3 * cosd(theta2 + theta3)];

    % Partial derivative with respect to theta3
    J3 = [-a3 * cosd(theta1) * sind(theta2 + theta3);
         -a3 * sind(theta1) * sind(theta2 + theta3);
          a3 * cosd(theta2 + theta3)];

    % Construct the Jacobian matrix
    J = [J1, J2, J3];
end
```